TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
KHOA CÔNG NGHỆ VÀ KỸ THUẬT

BÁO CÁO KẾT THÚC HỌC PHẦN

Học phần: Công nghệ blockchain và ứng dụng

Đề tài:

DAPP GÓP VỐN/ GÂY QUỸ SỬ DỤNG BLOCKCHAIN

Giảng viên hướng dẫn: Huỳnh Minh Châu

Sinh viên thực hiện: Huỳnh Thanh Duy
Nguyễn Duy Tường
Nguyễn Thị Kỳ Duyên
Đỗ Nguyễn Khiêm

Đồng Tháp, năm 2026

LỜI CAM ĐOAN

Chúng tôi xin cam đoan rằng báo cáo này là kết quả của quá trình nghiên cứu, thiết kế và phát triển hệ thống DApp gây quỹ (Crowdfunding) sử dụng công nghệ Blockchain do chính nhóm thực hiện dưới sự hướng dẫn của thầy Huỳnh Minh Châu.

Toàn bộ nội dung, kết quả thực nghiệm, mã nguồn Smart Contract cũng như giao diện ứng dụng được trình bày trong báo cáo là trung thực, được xây dựng từ quá trình tìm hiểu và thực hiện của nhóm, không sao chép từ bất kỳ công trình nào khác.

Các tài liệu tham khảo và kiến thức sử dụng trong quá trình thực hiện đều đã được trích dẫn rõ ràng và đầy đủ trong phần tài liệu tham khảo.

Chúng tôi xin hoàn toàn chịu trách nhiệm về tính chính xác và trung thực của nội dung báo cáo này.

P. Cao Lãnh, ngày 24 tháng 04 năm 2026

Tập thể nhóm thực hiện

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài "Xây dựng ứng dụng DApp gây quỹ (Crowdfunding) sử dụng công nghệ Blockchain", nhóm chúng tôi đã nhận được sự hướng dẫn và hỗ trợ quý báu từ thầy cô và bạn bè.

Trước hết, nhóm xin bày tỏ lòng biết ơn sâu sắc đến thầy Huỳnh Minh Châu – người đã tận tình hướng dẫn, định hướng và truyền đạt những kiến thức quan trọng về Blockchain, Smart Contract và phát triển ứng dụng phi tập trung (DApp). Những góp ý và chỉ dẫn của thầy là nền tảng quan trọng giúp nhóm hoàn thành đề tài.

Bên cạnh đó, nhóm cũng xin chân thành cảm ơn các thành viên trong nhóm đã cùng nhau nỗ lực, hợp tác và hỗ trợ lẫn nhau trong suốt quá trình nghiên cứu, xây dựng và kiểm thử hệ thống.

Mặc dù đã cố gắng hoàn thiện, báo cáo chắc chắn không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp từ quý thầy cô để đề tài được hoàn thiện hơn trong tương lai.

P. Cao Lãnh, ngày 24 tháng 04 năm 2026

Tập thể nhóm thực hiện

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Thành viên	Nhiệm vụ
1	Nguyễn Duy Tường	Xây dựng Smart Contract, thiết kế logic hệ thống, triển khai (deploy), kiểm thử hợp đồng, xử lý dữ liệu từ Smart Contract, tối ưu chi phí gas và xử lý lỗi giao dịch
2	Huỳnh Thanh Duy	Phụ trách phát triển giao diện (NextJS), tích hợp hệ thống, kết nối Blockchain, xử lý logic phía frontend và tương tác với Smart Contract
3	Nguyễn Thị Kỳ Duyên	Xây dựng kịch bản test, thiết kế hệ thống, thực hiện kiểm thử hệ thống, tổng hợp nội dung và hạn chế, phân tích và đánh giá kết quả
4	Đỗ Nguyễn Khiêm	Phân tích yêu cầu, viết đánh giá kết quả, viết kết luận và chuẩn hóa tài liệu và đề xuất hướng phát triển cho hệ thống

MỤC LỤC

Lời cam đoan .....

Lời cảm ơn ii

Mục lục iii

Danh mục từ viết tắt vi

Danh mục ký hiệu vii

Danh mục hình ảnh và đồ thị viii

Mở đầu 1

Chương 1. CƠ SỞ LÝ THUYẾT 4

1.1. Tổng quan về Blockchain 4

1.2. Nền tảng Ethereum 5

1.3. Smart Contract 5

1.4. Ứng dụng phi tập trung (DApp) 6

1.5. Công cụ và công nghệ sử dụng 6

1.6. Ngôn ngữ Solidity 7

1.7. Ví điện tử MetaMask 7

1.8. Thư viện ethers.js 8

1.9. Thư viện ReactJS 8

1.10. Framework Next.js 8

1.11. Kết luận chương 9

Chương 2. PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG 10

2.1. Phân tích bài toán 10

2.2. Yêu cầu hệ thống 10

2.2.1. Yêu cầu chức năng 10

2.2.2. Yêu cầu phi chức năng 13

2.3. Sơ đồ Use Case 13

2.3.1. Đặc tả 13

2.4. Thiết kế kiến trúc hệ thống **14**
2.5. Thiết kế Smart Contract **15**
2.5.1. Cấu trúc dữ liệu 15
2.5.2. Xử lý lỗi và kiểm tra dữ liệu 16
2.6. Kết luận chương **16**
Chương 3. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG **17**
3.1. Giới thiệu **17**
3.2. Xây dựng Smart Contract **17**
3.2.1. Giới thiệu về Solidity và môi trường phát triển 17
3.2.2. Thiết kế Smart Contract Crowdfunding 18
3.2.3. Các chức năng chính của Smart Contract 18
3.3. Triển khai Smart Contract bằng Hardhat **19**
3.4. Xây dựng giao diện người dùng **20**
3.4.1. Công nghệ sử dụng 20
3.4.2. Các chức năng giao diện 20
3.4.3. Giao diện hệ thống 20
3.5. Kết nối với Blockchain bằng Ethers.js **25**
3.6. Kiểm thử hệ thống **26**
3.7. Đánh giá triển khai **26**
3.8. Kết luận chương **27**
Chương 4. KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG **28**
4.1. Môi trường và phương pháp kiểm thử **28**
4.1.1. Môi trường kiểm thử 28
4.1.2. Phương pháp kiểm thử 28
4.2. Xây dựng kịch bản kiểm thử **29**
4.2.1. Kịch bản 1: Tạo chiến dịch 29
4.2.2. Kịch bản 2: Góp vốn 29
4.2.3. Kịch bản 3: Rút tiền (Trường hợp thành công) 30
4.2.4. Kịch bản 4: Hoàn tiền (Trường hợp thất bại) 30
4.3. Phân tích kết quả kiểm thử **30**
4.4. Đánh giá hệ thống **31**
4.4.1. Ưu điểm 31
4.4.2. Hạn chế 31
4.5. Hướng phát triển **31**
4.6. Kết luận chương **32**
KẾT LUẬN **32**

TÀI LIỆU THAM KHẢO **35**

DANH MỤC TỪ VIẾT TẮT

Viết tắt	**Thuật ngữ Tiếng Anh**	**Diễn giải Tiếng Việt**
DApp	Decentralized Application	Ứng dụng phi tập trung
Blockchain	Blockchain	Chuỗi khối dữ liệu phân tán
Smart Contract	Smart Contract	Hợp đồng thông minh
ETH	Ether	Tiền điện tử Ethereum
EVM	Ethereum Virtual Machine	Môi trường chạy contract
Gas	Gas	Phí giao dịch
IDE	Development Environment	Môi trường lập trình
UI	User Interface	Giao diện người dùng
UX	User Experience	Trải nghiệm người dùng
API	Application Interface	Giao diện lập trình
Web3	Web3	Web phi tập trung
MetaMask	MetaMask	Ví kết nối blockchain
Remix	Remix IDE	Công cụ viết contract
Solidity	Solidity	Ngôn ngữ smart contract
ReactJS	ReactJS	Thư viện giao diện web
ethers.js	ethers.js	Thư viện kết nối Ethereum
Frontend	Frontend	Giao diện hệ thống
Backend	Backend	Xử lý logic
Testnet	Test Network	Mạng thử nghiệm
Mainnet	Main Network	Mạng chính
Crowdfunding	Crowdfunding	Gây quỹ cộng đồng
Campaign	Campaign	Chiến dịch
Pledge	Pledge	Góp vốn
Claim	Claim	Rút tiền
Refund	Refund	Hoàn tiền

DANH MỤC KÝ HIỆU

Ký hiệu	**Diễn giải**
ETH	Đơn vị tiền điện tử dùng trong hệ thống Ethereum
$campaignId$	Mã định danh của một chiến dịch gây quỹ
$goal$	Mục tiêu số tiền cần đạt của chiến dịch
$pledged$	Tổng số tiền đã được góp vào chiến dịch
$startAt$	Thời điểm bắt đầu chiến dịch
$endAt$	Thời điểm kết thúc chiến dịch
$claimed$	Trạng thái đã rút tiền (true/false)
$msg.sender$	Địa chỉ người thực hiện giao dịch
$msg.value$	Số tiền ETH gửi trong giao dịch
$mapping$	Cấu trúc lưu trữ dữ liệu dạng ánh xạ
$address$	Địa chỉ ví trên blockchain
$uint256$	Kiểu dữ liệu số nguyên không âm

DANH MỤC HÌNH ẢNH VÀ ĐỒ THỊ

3.1 Giao diện trang chủ hệ thống 21
3.2 Giao diện tạo chiến dịch 22
3.3 Giao diện nhập số tiền góp vốn 23
3.4 Giao diện tìm kiếm và hiển thị danh sách chiến dịch 24
3.5 Giao diện xác nhận giao dịch góp vốn 25

MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh công nghệ số phát triển mạnh mẽ, đặc biệt là sự bùng nổ của công nghệ Blockchain, các ứng dụng phi tập trung (DApp) ngày càng được quan tâm và ứng dụng rộng rãi trong nhiều lĩnh vực như tài chính, thương mại và quản lý dữ liệu. Một trong những ứng dụng nổi bật của Blockchain là lĩnh vực gây quỹ cộng đồng (Crowdfunding), nơi người dùng có thể đóng góp tài chính cho các dự án mà không cần thông qua bên trung gian.

Trong thực tế, các nền tảng gây quỹ truyền thống thường tồn tại nhiều hạn chế như thiếu minh bạch trong quản lý dòng tiền, phụ thuộc vào bên trung gian và tiềm ẩn rủi ro gian lận. Người đóng góp thường không có cơ chế kiểm chứng rõ ràng về việc sử dụng nguồn vốn, trong khi người tạo dự án cũng gặp khó khăn trong việc xây dựng niềm tin với cộng đồng.

Công nghệ Blockchain với đặc tính minh bạch, bất biến và phi tập trung đã mở ra hướng tiếp cận mới cho bài toán này. Việc áp dụng Smart Contract giúp tự động hóa các quy trình như nhận tiền, kiểm tra điều kiện và hoàn tiền khi không đạt mục tiêu, từ đó giảm thiểu sự can thiệp của con người và nâng cao độ tin cậy của hệ thống.

Xuất phát từ những vấn đề thực tiễn đó, nhóm chúng tôi lựa chọn đề tài "Xây dựng ứng dụng DApp gây quỹ (Crowdfunding) sử dụng công nghệ Blockchain" nhằm xây dựng một hệ thống minh bạch, an toàn và tự động trong việc quản lý hoạt động góp vốn. Đồng thời, đề tài cũng giúp nhóm tiếp cận và vận dụng các kiến thức về Smart Contract, Web3 và phát triển ứng dụng phi tập trung.

2. Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng một ứng dụng DApp gây quỹ (Crowdfunding) hoạt động trên nền tảng Blockchain, đảm bảo tính minh bạch, an toàn và tự động trong quá trình quản lý nguồn vốn.

Cụ thể, hệ thống hướng đến các chức năng chính sau:

- Cho phép người dùng khởi tạo các chiến dịch gây quỹ với mục tiêu và thời gian xác định.

- Hỗ trợ người dùng tham gia đóng góp tài chính vào các chiến dịch một cách thuận tiện và minh bạch.
- Cho phép chủ chiến dịch rút tiền khi đạt đủ mục tiêu đề ra.
- Đảm bảo hoàn trả số tiền đã đóng góp cho người dùng trong trường hợp chiến dịch không thành công.
- Xây dựng giao diện thân thiện, dễ sử dụng và tích hợp kết nối với ví điện tử như MetaMask để thực hiện giao dịch.

3. Đối tượng và phạm vi đề tài

a) Đối tượng: Hệ thống ứng dụng DApp gây quỹ sử dụng Smart Contract viết bằng Solidity, triển khai trên mạng Blockchain (Ethereum hoặc mạng thử nghiệm như Sepolia), kết hợp với giao diện web sử dụng NextJS và thư viện ethers.js.

b) Phạm vi:

- Xây dựng và triển khai Smart Contract phục vụ chức năng gây quỹ.
- Phát triển giao diện web để người dùng tương tác với hệ thống.
- Kết nối ví MetaMask để thực hiện các giao dịch.
- Triển khai trên mạng thử nghiệm (testnet), không triển khai trên mạng chính (mainnet).
- Không đi sâu vào các vấn đề bảo mật nâng cao hoặc tối ưu chi phí gas.

4. Phương pháp thực hiện

Để hoàn thành đề tài, nhóm sử dụng các phương pháp sau:

a) Nghiên cứu tài liệu:

- Tìm hiểu công nghệ Blockchain, Ethereum và Smart Contract.
- Nghiên cứu ngôn ngữ Solidity và cách triển khai hợp đồng trên Remix.
- Tham khảo tài liệu về Web3, ethers.js và NextJS.

b) Thiết kế hệ thống:

- Thiết kế cấu trúc Smart Contract (Campaign, pledge, claim, refund).
- Xây dựng luồng hoạt động của hệ thống gây quỹ.

- Thiết kế giao diện người dùng.

c) Lập trình và triển khai:

- Viết và triển khai Smart Contract trên VScode.
- Xây dựng frontend bằng NextJS và kết nối blockchain bằng ethers.js.
- Kiểm thử chức năng bằng MetaMask và mạng testnet.

5. Bố cục báo cáo

Dựa vào mục tiêu và phương pháp nghiên cứu đã trình bày ở trên, báo cáo được bố cục gồm 3 phần chính: Mở đầu, Nội dung và Kết luận. Trong đó phần nội dung là trọng tâm, được chia thành 4 chương cụ thể như sau:

- **Chương 1: "Cơ sở lý thuyết"** Trình bày các kiến thức nền tảng gồm Blockchain, Ethereum, Smart Contract và DApp. Đồng thời giới thiệu các công cụ sử dụng như Solidity, Remix IDE, MetaMask, ethers.js và NextJS làm cơ sở xây dựng hệ thống.
- **Chương 2: "Phân tích yêu cầu và thiết kế hệ thống"** Phân tích bài toán gây quỹ, xác định các yêu cầu chức năng (tạo chiến dịch, góp vốn, rút tiền, hoàn tiền) và phi chức năng. Đồng thời thiết kế kiến trúc tổng thể và cấu trúc Smart Contract.
- **Chương 3: "Xây dựng và triển khai hệ thống"** Trình bày quá trình viết và triển khai Smart Contract trên Remix, xây dựng giao diện bằng ReactJS và kết nối với blockchain thông qua MetaMask và ethers.js.
- **Chương 4: "Kiểm thử và đánh giá"** Thực hiện kiểm thử với các kịch bản thực tế, đánh giá kết quả hoạt động của hệ thống và đề xuất hướng phát triển trong tương lai.

Chương 1

CƠ SỞ LÝ THUYẾT

Chương này trình bày các kiến thức nền tảng phục vụ cho việc xây dựng hệ thống ứng dụng gây quỹ (Crowdfunding) trên nền tảng Blockchain. Nội dung bao gồm tổng quan về công nghệ Blockchain, nền tảng Ethereum, Smart Contract và mô hình ứng dụng phi tập trung (DApp). Đây là cơ sở quan trọng giúp triển khai hệ thống một cách minh bạch, an toàn và hiệu quả.

1.1 Tổng quan về Blockchain

Blockchain là công nghệ lưu trữ và truyền tải dữ liệu dưới dạng các khối (block) được liên kết với nhau thông qua hàm băm (hash), tạo thành một chuỗi liên tục gọi là chuỗi khối (blockchain) [6]. Mỗi khối chứa thông tin giao dịch và được bảo vệ bằng cơ chế mật mã học, đảm bảo tính toàn vẹn và không thể bị thay đổi.

Các đặc điểm nổi bật của Blockchain:

- **Phi tập trung (Decentralization):** Dữ liệu không được lưu trữ tại một máy chủ trung tâm mà được phân tán trên nhiều node trong mạng.
- **Minh bạch (Transparency):** Mọi giao dịch đều được ghi nhận công khai và có thể kiểm tra.
- **Bất biến (Immutability):** Dữ liệu sau khi ghi vào blockchain không thể chỉnh sửa hoặc xóa bỏ.
- **Bảo mật cao (Security):** Áp dụng các thuật toán mã hóa và cơ chế đồng thuận giúp bảo vệ dữ liệu.

Nhờ những đặc điểm trên, Blockchain trở thành giải pháp phù hợp cho các hệ thống yêu cầu tính minh bạch và tin cậy cao như tài chính, chuỗi cung ứng và đặc biệt là các nền tảng gây quỹ [5].

1.2 Nền tảng Ethereum

Ethereum là một nền tảng Blockchain mã nguồn mở cho phép xây dựng và triển khai các ứng dụng phi tập trung (DApp) [6]. Điểm khác biệt của Ethereum so với các Blockchain khác là khả năng lập trình thông qua Smart Contract.

Các thành phần chính của Ethereum:

- **Ether (ETH):** Đồng tiền điện tử được sử dụng để thanh toán phí giao dịch.
- **Gas:** Chi phí thực thi các thao tác trên blockchain.
- **EVM (Ethereum Virtual Machine):** Môi trường thực thi Smart Contract.
- **Tài khoản (Account):** Bao gồm tài khoản người dùng (EOA) và tài khoản hợp đồng (Contract Account).

Ethereum là nền tảng phổ biến nhất hiện nay trong việc phát triển DApp nhờ hệ sinh thái phong phú và hỗ trợ mạnh mẽ từ cộng đồng [8].

1.3 Smart Contract

Smart Contract là các chương trình được triển khai trên blockchain, cho phép tự động thực thi các điều khoản đã được lập trình mà không cần sự can thiệp của bên trung gian [6].

Đặc điểm của Smart Contract:

- **Tự động thực thi:** Hoạt động theo logic đã được lập trình sẵn.
- **Không thể thay đổi:** Sau khi triển khai, nội dung hợp đồng gần như không thể chỉnh sửa.
- **Minh bạch:** Toàn bộ mã nguồn và hoạt động đều có thể kiểm tra công khai.

Trong hệ thống gây quỹ, Smart Contract đóng vai trò trung tâm với các chức năng chính [1]:

- Tạo và quản lý chiến dịch gây quỹ.
- Ghi nhận và lưu trữ thông tin góp vốn.
- Cho phép người tạo chiến dịch rút tiền khi đạt mục tiêu.
- Hoàn tiền cho người dùng nếu chiến dịch không thành công.

1.4 Ứng dụng phi tập trung (DApp)

DApp (Decentralized Application) là ứng dụng hoạt động trên nền tảng Blockchain, không phụ thuộc vào máy chủ trung tâm [6].

Cấu trúc của một DApp bao gồm:

- **Frontend:** Giao diện người dùng (thường sử dụng ReactJS).
- **Smart Contract:** Xử lý logic và dữ liệu trên blockchain.
- **Web3 Library:** Kết nối giữa frontend và blockchain (ethers.js).
- **Ví điện tử:** Cho phép người dùng tương tác (MetaMask).

DApp mang lại nhiều lợi ích như minh bạch, bảo mật và không bị kiểm soát bởi bên thứ ba, rất phù hợp cho hệ thống gây quỹ cộng đồng.

1.5 Công cụ và công nghệ sử dụng

Để xây dựng hệ thống, đề tài sử dụng các công cụ và công nghệ sau:

- **Solidity:** Ngôn ngữ lập trình Smart Contract trên nền tảng Ethereum [6].
- **Hardhat:** Framework hỗ trợ phát triển, biên dịch và triển khai Smart Contract trong môi trường Node.js.
- **Visual Studio Code:** Môi trường phát triển tích hợp (IDE) được sử dụng để viết và quản lý mã nguồn.
- **MetaMask:** Ví điện tử dùng để kết nối người dùng với blockchain và thực hiện các giao dịch [8].
- **Ethers.js:** Thư viện JavaScript giúp frontend tương tác với Smart Contract trên blockchain [8].
- **Next.js:** Framework React hỗ trợ xây dựng giao diện web với hiệu suất cao và khả năng render linh hoạt.
- **Infura (RPC Provider):** Dịch vụ cung cấp endpoint để kết nối frontend và backend tới mạng Ethereum (Sepolia testnet).

Các công cụ này kết hợp với nhau tạo thành một hệ sinh thái hoàn chỉnh, hỗ trợ phát triển ứng dụng DApp gây quỹ với khả năng mở rộng và tích hợp cao.

1.6 Ngôn ngữ Solidity

- **Solidity** là ngôn ngữ lập trình được sử dụng để viết Smart Contract trên Ethereum. Đây là ngôn ngữ hướng đối tượng, cú pháp tương tự JavaScript.

Các đặc điểm của Solidity:

- Hỗ trợ kiểu dữ liệu đa dạng (`uint`, `address`, `mapping`,...).
- Cho phép định nghĩa hàm, sự kiện (`event`).
- Tương tác trực tiếp với blockchain.

Trong đề tài này, Solidity được sử dụng để xây dựng các chức năng:

- `createCampaign()`: Khởi tạo chiến dịch mới.
- `pledge()`: Thực hiện góp vốn vào chiến dịch.
- `claim()`: Chủ chiến dịch rút tiền khi thành công.
- `refund()`: Hoàn tiền cho người đóng góp khi chiến dịch thất bại.

1.7 Ví điện tử MetaMask

- **MetaMask** là một tiện ích mở rộng trình duyệt đóng vai trò như ví điện tử, cho phép người dùng tương tác với blockchain Ethereum.

Chức năng:

- Lưu trữ khóa cá nhân (private key).
- Ký giao dịch.
- Kết nối với DApp.
- Chuyển/nhận ETH.

Trong hệ thống, MetaMask được sử dụng để:

- Kết nối người dùng.
- Thực hiện góp vốn.
- Xác nhận giao dịch.

1.8 Thư viện ethers.js

- **ethers.js** là một thư viện JavaScript dùng để tương tác với blockchain Ethereum.

Chức năng chính:

- Kết nối với node Ethereum.
- Gọi các hàm trong Smart Contract.
- Gửi giao dịch.
- Lắng nghe sự kiện (event).

`ethers.js` đóng vai trò là cầu nối giữa frontend và Smart Contract.

1.9 Thư viện ReactJS

- **ReactJS** là một thư viện JavaScript dùng để xây dựng giao diện người dùng (UI).

Ưu điểm:

- **Component-based:** Chia nhỏ giao diện thành các thành phần độc lập.
- **Tái sử dụng code:** Giúp quy trình phát triển tối ưu hơn.
- **Hiệu suất cao:** Nhờ cơ chế **Virtual DOM**.

1.10 Framework Next.js

- **Next.js** là một framework dựa trên React, được sử dụng để xây dựng ứng dụng web hiện đại với hiệu suất cao.

Ưu điểm:

- **Server-Side Rendering (SSR):** Giúp cải thiện hiệu suất và SEO.
- **Static Site Generation (SSG):** Tăng tốc độ tải trang.
- **Routing tự động:** Quản lý điều hướng dễ dàng.
- **Tối ưu hiệu suất:** Tự động tối ưu hình ảnh và tài nguyên.

Trong đề tài:

- Sử dụng để xây dựng giao diện DApp.
- Kết hợp với `ethers.js` để hiển thị dữ liệu từ Smart Contract.

1.11 Kết luận chương

Chương này đã trình bày các kiến thức nền tảng và công cụ cần thiết cho việc xây dựng hệ thống DApp Crowdfunding. Các công nghệ như Blockchain, Ethereum và Smart Contract đóng vai trò cốt lõi trong việc đảm bảo tính minh bạch và tự động hóa của hệ thống. Bên cạnh đó, các công cụ như Solidity, Remix IDE, MetaMask, ethers.js và ReactJS hỗ trợ quá trình phát triển và triển khai ứng dụng một cách hiệu quả.

Trong đề tài này, các công nghệ trên được áp dụng để xây dựng hệ thống gây quỹ thực tế, cho phép người dùng tạo chiến dịch, góp vốn và theo dõi trạng thái trực tiếp thông qua giao diện web và ví MetaMask trên mạng Sepolia.

Chương 2

PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích bài toán

Trong những năm gần đây, mô hình gây quỹ cộng đồng (Crowdfunding) đã trở thành một phương thức phổ biến giúp các cá nhân hoặc tổ chức huy động vốn cho các dự án sáng tạo, khởi nghiệp hoặc hoạt động xã hội [6]. Tuy nhiên, các nền tảng gây quỹ truyền thống vẫn tồn tại nhiều hạn chế như phụ thuộc vào bên trung gian, thiếu minh bạch trong quản lý dòng tiền và tiềm ẩn nguy cơ gian lận.

Cụ thể, người đóng góp không có khả năng kiểm chứng việc sử dụng nguồn vốn sau khi đã chuyển tiền, trong khi người tạo dự án phải phụ thuộc vào uy tín nền tảng để thu hút nhà đầu tư. Ngoài ra, chi phí trung gian và quy trình xử lý thủ công cũng làm giảm hiệu quả hoạt động của hệ thống.

Với sự phát triển của công nghệ Blockchain, đặc biệt là Smart Contract, các vấn đề trên có thể được giải quyết một cách hiệu quả [5]. Blockchain cho phép lưu trữ dữ liệu minh bạch, không thể thay đổi, trong khi Smart Contract giúp tự động hóa các điều kiện giao dịch như nhận tiền, kiểm tra mục tiêu và hoàn tiền.

Do đó, việc xây dựng một hệ thống DApp gây quỹ là giải pháp phù hợp nhằm nâng cao tính minh bạch, giảm thiểu rủi ro và loại bỏ sự phụ thuộc vào bên trung gian.

2.2 Yêu cầu hệ thống

2.2.1 Yêu cầu chức năng

Hệ thống DApp gây quỹ cần đáp ứng các chức năng chính sau:

a) Tạo chiến dịch (createCampaign)

Cho phép người dùng tạo một chiến dịch gây quỹ với các thông tin:

- Tên chiến dịch (tối đa 32 ký tự)
- Mục tiêu tài chính (ETH)
- Thời gian kêu gọi (tính theo ngày)

Ngoài ra, hệ thống có các ràng buộc:

- Mỗi người dùng chỉ được tạo tối đa 5 chiến dịch đang hoạt động.
- Thời gian kết thúc được tính tự động dựa trên thời điểm tạo.

b) Góp vốn (pledge)

Người dùng có thể đóng góp ETH vào các chiến dịch:

- Chỉ thực hiện khi chiến dịch chưa hết hạn.
- Số tiền đóng góp được lưu trên blockchain.
- Hệ thống ghi nhận tổng số tiền đã huy động và số tiền từng người đóng góp.

c) Rút tiền (claim)

Người tạo chiến dịch có thể rút tiền khi:

- Chiến dịch đạt mục tiêu tài chính.
- Chưa từng rút tiền trước đó.

Khi thực hiện:

- Toàn bộ số ETH sẽ được chuyển về ví của người tạo.
- Trạng thái chiến dịch chuyển sang "đã hoàn thành".

d) Hoàn tiền (refund)

Người dùng có thể yêu cầu hoàn tiền khi:

- Chiến dịch đã kết thúc.

- Không đạt mục tiêu tài chính.

Khi thực hiện:

- Người dùng nhận lại đúng số ETH đã đóng góp.
- Mỗi người chỉ hoàn tiền 1 lần.

e) Kết nối ví (MetaMask)

Hệ thống hỗ trợ kết nối ví MetaMask để:

- Xác thực người dùng.
- Thực hiện các giao dịch như tạo chiến dịch, góp vốn, rút tiền, hoàn tiền.

Ngoài ra dữ liệu chiến dịch vẫn có thể được đọc thông qua RPC provider (Infura) mà không cần kết nối ví và việc kết nối ví chỉ bắt buộc khi thực hiện giao dịch.

f) Hiển thị danh sách chiến dịch

Hệ thống cho phép xem toàn bộ các chiến dịch đã tạo và hiển thị thông tin: tên, người tạo, mục tiêu, số tiền đã góp, thời gian còn lại, trạng thái.

g) Xem chi tiết chiến dịch

Người dùng có thể xem chi tiết:

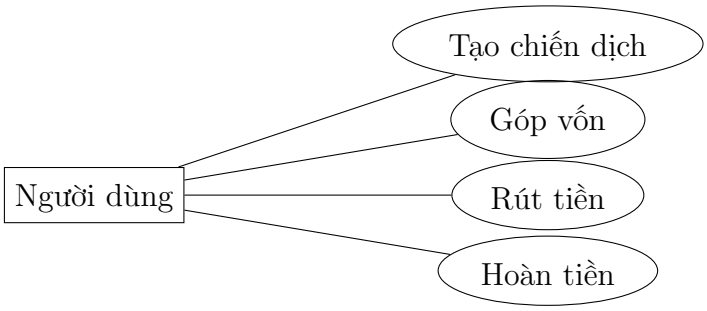
- Thông tin đầy đủ của chiến dịch.
- Lịch sử đóng góp (event Pledged).
- Lịch sử rút tiền (event Claimed).
- Lịch sử hoàn tiền (event Refunded).

2.2.2 Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cần đảm bảo các yêu cầu sau [1]:

- **Tính minh bạch:** Mọi giao dịch đều được ghi nhận trên Blockchain và có thể kiểm tra công khai.
- **Tính bảo mật:** Dữ liệu không thể bị chỉnh sửa hoặc giả mạo sau khi được xác nhận.
- **Tính tin cậy:** Các quy trình được tự động hóa bằng Smart Contract, hạn chế tối đa sự can thiệp của con người [6].
- **Tính dễ sử dụng:** Giao diện trực quan, thân thiện với người dùng.
- **Hiệu suất:** Hệ thống hoạt động ổn định, đảm bảo tốc độ xử lý giao dịch.

2.3 Sơ đồ Use Case



Mô tả Use Case: Người dùng đóng vai trò trung tâm của hệ thống, có thể thực hiện tất cả các chức năng chính bao gồm tạo chiến dịch, tham gia góp vốn, rút tiền khi thành công và hoàn tiền khi thất bại. Các chức năng này được thực hiện thông qua giao diện web và được xử lý bởi Smart Contract trên Blockchain [5].

2.3.1 Đặc tả

Use Case 1: Tạo chiến dịch

- Actor: Người dùng
- Mô tả: Tạo chiến dịch gây quỹ mới
- Điều kiện: Đã kết nối ví MetaMask
- Kết quả: Chiến dịch được lưu trên Blockchain

Use Case 2: Góp vốn

- Actor: Người dùng
- Mô tả: Góp ETH vào chiến dịch
- Điều kiện: Chiến dịch đang hoạt động
- Kết quả: Số tiền được cập nhật

Use Case 3: Rút tiền

- Actor: Chủ chiến dịch
- Điều kiện: Đạt mục tiêu
- Kết quả: ETH được chuyển về ví

Use Case 4: Hoàn tiền

- Actor: Người góp vốn
- Điều kiện: Không đạt mục tiêu
- Kết quả: Nhận lại ETH

2.4 Thiết kế kiến trúc hệ thống

Hệ thống DApp gây quỹ được xây dựng theo mô hình kiến trúc 3 lớp [6]:

- **Frontend (Giao diện):** Được xây dựng bằng NextJS, cho phép người dùng tương tác với hệ thống thông qua trình duyệt.
- **Blockchain (Smart Contract):** Là nơi xử lý logic nghiệp vụ, lưu trữ dữ liệu và thực thi các giao dịch.
- **Wallet (MetaMask):** Đóng vai trò trung gian kết nối giữa người dùng và Blockchain, dùng để ký và xác nhận giao dịch.

Luồng hoạt động: Người dùng thao tác trên giao diện → gửi yêu cầu thông qua MetaMask → Smart Contract xử lý và ghi nhận trên Blockchain → kết quả trả về giao diện.

Luồng hoạt động hệ thống:

1. Người dùng truy cập giao diện web
2. Kết nối ví MetaMask

3. Thực hiện thao tác (tạo chiến dịch, góp vốn,...)
4. MetaMask yêu cầu xác nhận giao dịch
5. Smart Contract xử lý và ghi dữ liệu lên Blockchain
6. Kết quả hiển thị lại trên giao diện

2.5 Thiết kế Smart Contract

Smart Contract là thành phần cốt lõi của hệ thống [1], chịu trách nhiệm quản lý toàn bộ logic của quá trình gây quỹ trên blockchain. Hợp đồng được xây dựng bằng ngôn ngữ Solidity và triển khai trên mạng Ethereum (Sepolia testnet), với các cấu trúc và chức năng chính như sau:

- **Struct Campaign:** Lưu thông tin chiến dịch gồm người tạo, mục tiêu, số tiền đã góp, thời gian và trạng thái.
- **Mapping:** Lưu trữ danh sách chiến dịch và số tiền mà mỗi người đã đóng góp.
- **Hàm createCampaign():** Khởi tạo chiến dịch mới với các điều kiện hợp lệ.
- **Hàm pledge():** Cho phép người dùng gửi ETH vào hợp đồng.
- **Hàm claim():** Cho phép chủ chiến dịch rút tiền khi đạt mục tiêu.
- **Hàm refund():** Hoàn tiền cho người dùng nếu chiến dịch thất bại.
- **Hàm openCampaignCount():** Hàm hỗ trợ kiểm tra số lượng chiến dịch đang hoạt động của một người dùng:

Các hàm đều được kiểm tra điều kiện bằng `require()` nhằm đảm bảo tính chính xác và an toàn trong quá trình thực thi.

2.5.1 Cấu trúc dữ liệu

- Cấu trúc Campaign:
  - creator: địa chỉ ví của người tạo chiến dịch
  - name: tên chiến dịch
  - goal: mục tiêu gây quỹ
  - pledged: tổng số tiền đã được đóng góp
  - deadline: thời điểm kết thúc chiến dịch (timestamp)

  - claimed: trạng thái rút tiền

- Mapping:
  - campaigns[id]: lưu thông tin chiến dịch theo ID.
  - pledgedAmount[user][campaignId]: lưu số tiền mà mỗi người dùng đã đóng góp vào từng chiến dịch.

- Biến đếm chiến dịch:
  - count: lưu tổng số chiến dịch đã được tạo.

2.5.2 Xử lý lỗi và kiểm tra dữ liệu

Để đảm bảo hệ thống hoạt động ổn định, các điều kiện kiểm tra được áp dụng:

- Không cho phép tên chiến dịch rỗng hoặc vượt quá 32 ký tự.
- Giới hạn mỗi người chỉ được tạo tối đa 5 chiến dịch đang hoạt động.
- Không cho phép góp vốn khi chiến dịch đã hết hạn.
- Chỉ người tạo chiến dịch mới được rút tiền.
- Không cho phép rút tiền khi chưa đạt mục tiêu.
- Không cho phép rút tiền nhiều lần.
- Chỉ cho phép hoàn tiền khi chiến dịch đã kết thúc.
- Không cho phép hoàn tiền nếu chiến dịch đã đạt mục tiêu.

2.6 Kết luận chương

Chương này đã trình bày chi tiết bài toán gây quỹ, phân tích các yêu cầu chức năng và phi chức năng, đồng thời xây dựng thiết kế tổng thể của hệ thống thông qua sơ đồ Use Case, kiến trúc hệ thống và cấu trúc Smart Contract. Đây là nền tảng quan trọng để triển khai và phát triển hệ thống trong các chương tiếp theo.

Chương 3

XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

3.1 Giới thiệu

Sau khi hoàn thành quá trình phân tích yêu cầu và thiết kế hệ thống, bước tiếp theo là tiến hành xây dựng và triển khai ứng dụng DApp gây quỹ trên nền tảng blockchain. Đây là giai đoạn quan trọng nhằm hiện thực hóa các ý tưởng và mô hình đã đề xuất ở chương trước.

Chương này trình bày chi tiết quá trình phát triển Smart Contract bằng ngôn ngữ Solidity trong môi trường Visual Studio Code, sử dụng framework Hardhat để biên dịch và triển khai hợp đồng lên mạng Ethereum (Sepolia testnet) thông qua RPC provider (Infura). Bên cạnh đó, chương cũng mô tả việc xây dựng giao diện người dùng bằng Next.js và thực hiện kết nối với blockchain thông qua ví MetaMask và thư viện Ethers.js.

3.2 Xây dựng Smart Contract

3.2.1 Giới thiệu về Solidity và môi trường phát triển

Solidity là ngôn ngữ lập trình hướng đối tượng được thiết kế chuyên biệt để viết Smart Contract trên nền tảng Ethereum [6]. Ngôn ngữ này cho phép định nghĩa các cấu trúc dữ liệu, hàm xử lý và các điều kiện logic nhằm kiểm soát luồng giao dịch trên Blockchain.

Trong đề tài này, Smart Contract được phát triển bằng môi trường Visual Studio Code kết hợp với framework Hardhat. Hardhat hỗ trợ biên dịch, kiểm thử và triển khai Smart Contract một cách hiệu quả, đồng thời tích hợp tốt với thư viện Ethers.js để tương tác với blockchain.

Việc sử dụng Visual Studio Code giúp tăng tính linh hoạt trong quá trình phát triển, cho phép quản lý mã nguồn, sử dụng các extension hỗ trợ lập trình và dễ dàng tích hợp với hệ thống Git để làm việc nhóm.

3.2.2 Thiết kế Smart Contract Crowdfunding

Hợp đồng Crowdfunding được xây dựng nhằm quản lý toàn bộ hoạt động gây quỹ, bao gồm tạo chiến dịch, nhận tiền đóng góp, rút tiền và hoàn tiền [5].

Cấu trúc dữ liệu chính:

- **Struct Campaign:** Lưu trữ thông tin của một chiến dịch gồm:
  - creator: địa chỉ ví của người tạo chiến dịch
  - name: tên chiến dịch (giới hạn độ dài)
  - goal: mục tiêu gây quỹ
  - pledged: tổng số tiền đã huy động
  - deadline: thời điểm kết thúc chiến dịch (timestamp)
  - claimed: trạng thái đã rút tiền hay chưa
- **Mapping:**
  - campaigns: lưu danh sách các chiến dịch theo ID
  - pledgedAmount: mapping hai chiều lưu số tiền mỗi người đã đóng góp cho từng chiến dịch (campaignId → user → amount)
- **Biến đếm:**
  - count: lưu tổng số chiến dịch đã được tạo, đồng thời dùng để sinh ID tự tăng

Ý nghĩa thiết kế: Việc sử dụng struct và mapping giúp lưu trữ dữ liệu một cách hiệu quả trên Blockchain, đồng thời đảm bảo truy xuất nhanh và rõ ràng [1].

3.2.3 Các chức năng chính của Smart Contract

- **Tạo chiến dịch (createCampaign):** Cho phép người dùng tạo chiến dịch mới với các thông tin cơ bản. Hệ thống kiểm tra điều kiện hợp lệ về thời gian và mục tiêu nhằm tránh sai sót.
- **Góp vốn (pledge):** Người dùng có thể gửi ETH vào hợp đồng để đóng góp cho chiến dịch. Số tiền được ghi nhận và cập nhật vào tổng quỹ.
- **Rút tiền (claim):** Khi chiến dịch kết thúc và đạt mục tiêu, người tạo chiến dịch có thể rút toàn bộ số tiền đã huy động.
- **Hoàn tiền (refund):** Nếu chiến dịch không đạt mục tiêu, người đóng góp có thể yêu cầu hoàn lại số tiền đã gửi.

Cơ chế đảm bảo an toàn: Các hàm đều sử dụng lệnh `require()` để kiểm tra điều kiện trước khi thực hiện. Điều này giúp ngăn chặn các giao dịch không hợp lệ và đảm bảo tính toàn vẹn của hệ thống [4].

3.3 Triển khai Smart Contract bằng Hardhat

Quá trình triển khai Smart Contract được thực hiện bằng framework Hardhat trong môi trường Visual Studio Code, bao gồm các bước sau:

- Tạo project Hardhat và cấu hình môi trường phát triển (Node.js, Hardhat, Ethers.js).
- Viết mã nguồn Smart Contract (Crowdfunding.sol) trong thư mục *contracts*.
- Tiến hành biên dịch Smart Contract bằng lệnh:

  ```
  npx hardhat compile
  ```

  Nếu không có lỗi, hệ thống sẽ tạo ra ABI và bytecode trong thư mục *artifacts*.
- Cấu hình mạng Sepolia Testnet trong file cấu hình (*hardhat.config.ts*), sử dụng RPC provider (Infura) và private key của ví MetaMask.
- Tạo script triển khai trong thư mục *scripts* (deploy.js) để tự động deploy Smart Contract.
- Thực hiện triển khai bằng lệnh:

  ```
  npx hardhat run scripts/deploy.js --network sepolia
  ```

- Xác nhận giao dịch được thực hiện thông qua ví MetaMask. Sau khi triển khai thành công, hệ thống sẽ trả về địa chỉ Smart Contract (Contract Address).
- Sử dụng Contract Address và ABI để tích hợp với frontend (Next.js) thông qua thư viện Ethers.js.

3.4 Xây dựng giao diện người dùng

3.4.1 Công nghệ sử dụng

Giao diện được xây dựng bằng Next.js, một framework dựa trên React, hỗ trợ Server-Side Rendering (SSR) và Static Site Generation (SSG), giúp tối ưu hiệu suất và nâng cao trải nghiệm người dùng. Thư viện ethers.js được sử dụng để kết nối và tương tác với Blockchain [6].

Vai trò trong hệ thống:

- Quản lý thư viện thông qua npm
- Chạy ứng dụng frontend
- Build và triển khai hệ thống

3.4.2 Các chức năng giao diện

- Kết nối ví MetaMask để xác thực người dùng
- Hiển thị danh sách các chiến dịch gây quỹ
- Tạo chiến dịch mới
- Góp vốn vào chiến dịch
- Thực hiện rút tiền hoặc hoàn tiền

Giao diện được thiết kế đơn giản, trực quan, đảm bảo người dùng có thể thao tác dễ dàng ngay cả khi không có kiến thức sâu về Blockchain.

3.4.3 Giao diện hệ thống

Giao diện hệ thống được xây dựng theo hướng đơn giản, trực quan, giúp người dùng dễ dàng thao tác với các chức năng chính như tạo chiến dịch, góp vốn và theo dõi trạng thái chiến dịch.

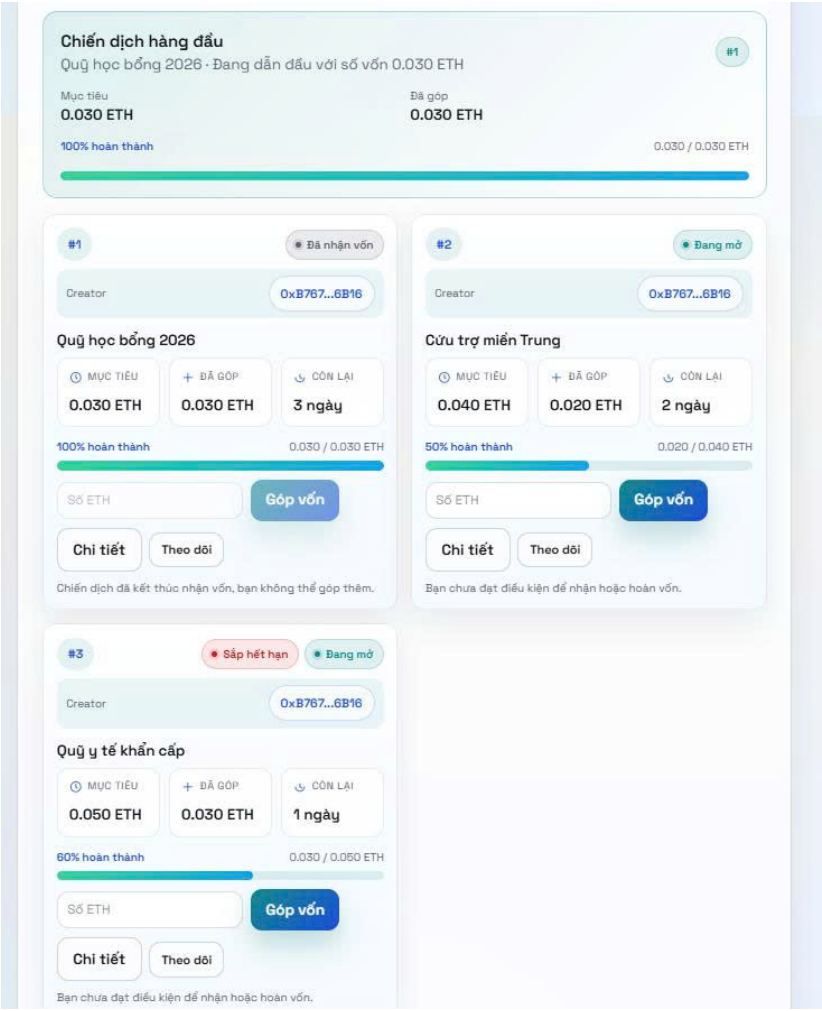
Hình 3.1. Giao diện trang chủ hệ thống

Giao diện trang chủ hiển thị danh sách các chiến dịch đang hoạt động, cho phép người dùng theo dõi thông tin tổng quan và lựa chọn chiến dịch để tham gia.

Ngoài ra, hệ thống hỗ trợ nhập tên chiến dịch vào ô tìm kiếm để lọc và hiển thị nhanh các kết quả phù hợp, giúp người dùng dễ dàng tra cứu và truy cập chiến dịch mong muốn.

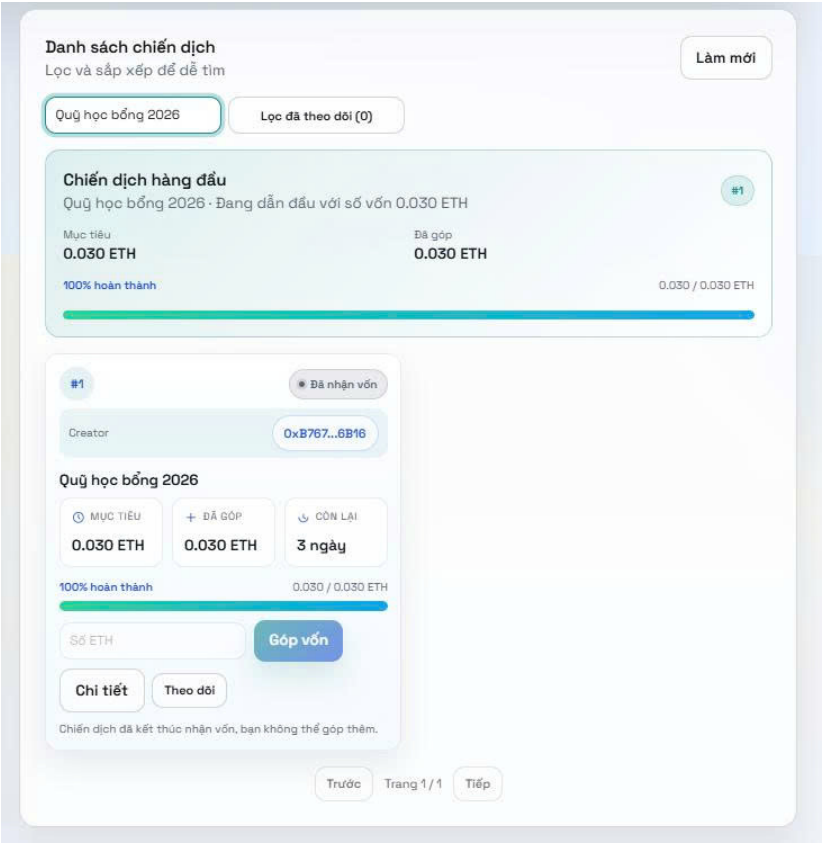
Hình 3.2. Giao diện tạo chiến dịch

Tại giao diện này, người dùng có thể nhập các thông tin cần thiết như mục tiêu gây quỹ, thời gian bắt đầu và kết thúc để khởi tạo chiến dịch mới.



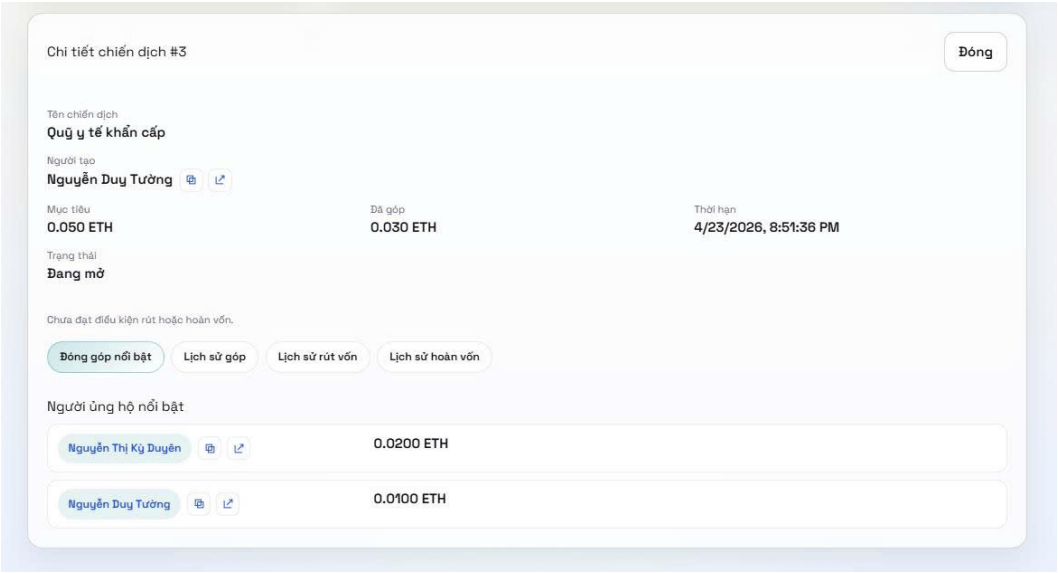
Hình 3.3. Giao diện nhập số tiền góp vốn

Người dùng nhập số tiền muốn đóng góp và thực hiện xác nhận giao dịch thông qua ví MetaMask.



Hình 3.4. Giao diện tìm kiếm và hiển thị danh sách chiến dịch

Tại giao diện này, người dùng có thể nhập từ khóa để tìm kiếm các chiến dịch gây quỹ theo tên hoặc nội dung liên quan. Kết quả tìm kiếm sẽ được hiển thị dưới dạng danh sách, giúp người dùng dễ dàng theo dõi, lựa chọn và tham gia góp vốn vào các chiến dịch phù hợp.



Hình 3.5. Giao diện xác nhận giao dịch góp vốn

Sau khi nhập số tiền, hệ thống hiển thị cửa sổ xác nhận giao dịch để người dùng kiểm tra và hoàn tất quá trình góp vốn.

3.5 Kết nối với Blockchain bằng Ethers.js

Frontend và Smart Contract được kết nối thông qua thư viện Ethers.js, cho phép ứng dụng web tương tác trực tiếp với blockchain Ethereum.

Quá trình kết nối được thực hiện như sau:

```
const provider = new ethers.BrowserProvider(window.ethereum);
await provider.send("eth_requestAccounts", []);
const signer = await provider.getSigner();
const contract = new ethers.Contract(contractAddress, abi, signer);
```

Trong đó:

- provider: kết nối tới blockchain thông qua MetaMask
- signer: đại diện cho tài khoản người dùng, dùng để ký giao dịch
- contract: đối tượng đại diện Smart Contract, cho phép gọi các hàm

Sau khi kết nối thành công, hệ thống có thể gọi các hàm trong Smart Contract như:

- createCampaign()

- pledge()
- claim()
- refund()

Mỗi giao dịch đều yêu cầu người dùng xác nhận trên ví MetaMask trước khi được gửi lên blockchain. Sau khi người dùng xác nhận, frontend sẽ theo dõi trạng thái giao dịch bằng cách chờ transaction được xác nhận on-chain (ví dụ: `await tx.wait()`).

3.6 Kiểm thử hệ thống

Hệ thống được kiểm thử thông qua các kịch bản thực tế nhằm đảm bảo hoạt động chính xác:

Kịch bản 1 – Thành công:

- Người dùng A tạo chiến dịch
- Người dùng B góp vốn đủ mục tiêu
- Sau thời gian kết thúc, A rút tiền thành công

Kịch bản 2 – Thất bại:

- Người dùng góp vốn nhưng không đạt mục tiêu
- Sau khi hết hạn, người dùng thực hiện hoàn tiền

Kết quả kiểm thử cho thấy hệ thống hoạt động đúng theo logic đã thiết kế, đảm bảo tính minh bạch và an toàn.

3.7 Đánh giá triển khai

Ưu điểm của hệ thống:

- Minh bạch, không thể chỉnh sửa dữ liệu
- Tự động hóa quy trình bằng Smart Contract
- Không phụ thuộc bên trung gian [6]

Hạn chế:

- Phụ thuộc vào mạng Blockchain (tốc độ, phí gas)
- Giao diện còn đơn giản
- Chưa tối ưu bảo mật nâng cao

3.8 Kết luận chương

Chương này đã trình bày toàn bộ quá trình xây dựng và triển khai hệ thống DApp gây quỹ, từ việc phát triển Smart Contract đến xây dựng giao diện và kết nối với Blockchain. Kết quả cho thấy hệ thống hoạt động đúng chức năng và đáp ứng các yêu cầu đề ra, tạo tiền đề cho việc đánh giá và phát triển trong chương tiếp theo.

Chương 4

KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

Sau khi hoàn thiện quá trình xây dựng và triển khai hệ thống DApp gây quỹ, việc kiểm thử và đánh giá là bước quan trọng nhằm xác minh tính đúng đắn của hệ thống, đảm bảo các chức năng hoạt động theo đúng thiết kế ban đầu và đáp ứng yêu cầu thực tế [8].

Chương này tập trung xây dựng các kịch bản kiểm thử bám sát luồng hoạt động thực tế của hệ thống, phân tích kết quả đạt được, đồng thời đánh giá ưu điểm, hạn chế và đề xuất các hướng phát triển nhằm hoàn thiện hệ thống trong tương lai.

4.1 Môi trường và phương pháp kiểm thử

4.1.1 Môi trường kiểm thử

Hệ thống được kiểm thử trong môi trường giả lập Blockchain với các thành phần sau [8]:

- Trình duyệt: Google Chrome
- Ví điện tử: MetaMask
- Mạng thử nghiệm: Sepolia Testnet
- Công cụ phát triển: Remix IDE, Visual Studio Code
- Thư viện kết nối: ethers.js

Việc sử dụng mạng testnet giúp đảm bảo tính an toàn, không phát sinh chi phí thực tế trong quá trình kiểm thử.

4.1.2 Phương pháp kiểm thử

Hệ thống được kiểm thử theo phương pháp kiểm thử chức năng (Functional Testing), dựa trên các kịch bản sử dụng thực tế của người dùng [4].

Ngoài ra, kiểm thử còn tập trung vào:

- Kiểm tra tính đúng đắn của logic Smart Contract
- Kiểm tra luồng giao dịch giữa frontend và Blockchain
- Đánh giá phản hồi của giao diện người dùng

4.2 Xây dựng kịch bản kiểm thử

4.2.1 Kịch bản 1: Tạo chiến dịch

Mô tả: Người dùng tạo một chiến dịch mới với mục tiêu và thời gian xác định.

Các bước thực hiện:

- Kết nối MetaMask
- Nhập thông tin chiến dịch
- Gửi giao dịch tạo chiến dịch

Kết quả mong đợi:

- Chiến dịch được tạo thành công
- Dữ liệu được lưu trên Blockchain

Kết quả thực tế: Hệ thống hoạt động đúng, chiến dịch hiển thị trên giao diện và có thể truy vấn từ Smart Contract.

4.2.2 Kịch bản 2: Góp vốn

Mô tả: Người dùng thực hiện góp vốn vào chiến dịch đang hoạt động.

Các bước thực hiện:

- Chọn chiến dịch
- Nhập số tiền góp
- Xác nhận giao dịch trên MetaMask

Kết quả mong đợi:

- Số tiền được cập nhật chính xác
- Tổng số tiền chiến dịch tăng tương ứng

Kết quả thực tế: Giao dịch được ghi nhận trên Blockchain, dữ liệu cập nhật đúng và hiển thị chính xác trên giao diện.

4.2.3 Kịch bản 3: Rút tiền (Trường hợp thành công)

Mô tả: Người tạo chiến dịch rút tiền khi chiến dịch đạt mục tiêu.

Điều kiện:

- Chiến dịch đã kết thúc
- Tổng tiền đạt hoặc vượt mục tiêu

Kết quả mong đợi:

- Toàn bộ số tiền được chuyển về ví người tạo
- Trạng thái chiến dịch cập nhật

Kết quả thực tế: Giao dịch thành công, tiền được chuyển chính xác, trạng thái "claimed" được cập nhật đúng.

4.2.4 Kịch bản 4: Hoàn tiền (Trường hợp thất bại)

Mô tả: Chiến dịch không đạt mục tiêu và người dùng yêu cầu hoàn tiền.

Điều kiện:

- Chiến dịch đã kết thúc
- Tổng tiền nhỏ hơn mục tiêu

Kết quả mong đợi:

- Người dùng nhận lại đúng số tiền đã góp

Kết quả thực tế: Hệ thống xử lý hoàn tiền chính xác, đảm bảo tính công bằng cho người dùng.

4.3 Phân tích kết quả kiểm thử

Qua các kịch bản kiểm thử, hệ thống đạt được các kết quả sau [4]:

- Tất cả các chức năng chính đều hoạt động đúng theo thiết kế
- Logic xử lý trong Smart Contract chính xác và ổn định
- Giao dịch được xác nhận rõ ràng thông qua MetaMask
- Không phát hiện lỗi nghiêm trọng trong quá trình kiểm thử

Điều này chứng tỏ hệ thống đã đáp ứng tốt yêu cầu ban đầu và có thể vận hành trong môi trường thực tế.

4.4 Đánh giá hệ thống

Dựa trên quá trình xây dựng và kiểm thử, hệ thống được đánh giá trên các tiêu chí về tính minh bạch, hiệu suất và khả năng mở rộng như sau:

4.4.1 Ưu điểm

- **Minh bạch:** Tất cả giao dịch được lưu trữ trên Blockchain và có thể kiểm tra công khai [6].
- **Tự động hóa:** Các điều kiện giao dịch được xử lý hoàn toàn bằng Smart Contract.
- **Không trung gian:** Loại bỏ sự phụ thuộc vào bên thứ ba, giảm chi phí và rủi ro [6].
- **Tính toàn vẹn dữ liệu:** Dữ liệu không thể bị thay đổi sau khi ghi nhận.

4.4.2 Hạn chế

- Phụ thuộc vào mạng Blockchain nên tốc độ xử lý có thể bị ảnh hưởng.
- Chi phí gas có thể tăng cao khi triển khai trên mạng chính [8].
- Giao diện chưa tối ưu, chưa có nhiều tính năng nâng cao.
- Chưa xử lý các trường hợp bảo mật phức tạp như tấn công hợp đồng [7].

4.5 Hướng phát triển

Để nâng cao chất lượng và khả năng ứng dụng thực tế, hệ thống có thể được phát triển theo các hướng sau:

- Cải thiện giao diện người dùng theo hướng hiện đại, thân thiện hơn (UI/UX).
- Tối ưu Smart Contract nhằm giảm chi phí gas và tăng hiệu suất [5].
- Tích hợp token ERC20 thay cho ETH để mở rộng khả năng sử dụng.
- Bổ sung cơ chế bảo mật nâng cao như kiểm tra quyền truy cập và chống tấn công [7].
- Triển khai hệ thống trên mạng chính (Mainnet) để đưa vào sử dụng thực tế.
- Xây dựng thêm các tính năng như đánh giá chiến dịch, bình luận, theo dõi tiến độ.

4.6 Kết luận chương

Chương này đã trình bày quá trình kiểm thử và đánh giá hệ thống DApp gây quỹ một cách toàn diện. Kết quả cho thấy hệ thống hoạt động ổn định, đáp ứng tốt các yêu cầu đề ra và đảm bảo tính minh bạch, an toàn.

Mặc dù vẫn còn một số hạn chế nhất định, nhưng với các hướng phát triển phù hợp, hệ thống hoàn toàn có thể được mở rộng và ứng dụng trong thực tế.

KẾT LUẬN

Trong bối cảnh công nghệ Blockchain ngày càng phát triển mạnh mẽ, việc xây dựng các ứng dụng phi tập trung (DApp) đã mở ra nhiều hướng đi mới trong việc giải quyết các bài toán về minh bạch và niềm tin, đặc biệt trong lĩnh vực gây quỹ cộng đồng (Crowdfunding) [6]. Qua quá trình nghiên cứu và triển khai đề tài "Xây dựng ứng dụng DApp gây quỹ sử dụng công nghệ Blockchain", nhóm đã đạt được một số kết quả nhất định.

a) Kết quả đạt được. Đề tài đã nghiên cứu và tổng hợp các kiến thức nền tảng về Blockchain, Ethereum, Smart Contract và DApp [5, 6], từ đó làm cơ sở để xây dựng hệ thống gây quỹ phi tập trung. Đồng thời, nhóm đã thiết kế và triển khai thành công một ứng dụng DApp với các chức năng cơ bản, cụ thể:

(1) Xây dựng Smart Contract bằng Solidity cho phép tạo chiến dịch gây quỹ, ghi nhận giao dịch góp vốn, thực hiện rút tiền khi đạt mục tiêu và hoàn tiền khi chiến dịch thất bại [1, 2]. Các chức năng được tự động hóa, đảm bảo tính minh bạch và không phụ thuộc vào bên trung gian.

(2) Phát triển giao diện người dùng bằng ReactJS, cho phép người dùng dễ dàng tương tác với hệ thống như tạo chiến dịch, tham gia góp vốn và theo dõi trạng thái chiến dịch.

(3) Kết nối hệ thống với blockchain thông qua thư viện ethers.js và ví MetaMask [8], giúp thực hiện các giao dịch một cách an toàn và trực quan.

(4) Tiến hành kiểm thử hệ thống trên mạng thử nghiệm (testnet), đảm bảo các chức năng hoạt động đúng logic, dữ liệu được lưu trữ minh bạch và các giao dịch được thực thi chính xác [4].

b) Hạn chế và hướng phát triển. Bên cạnh những kết quả đạt được, đề tài vẫn còn một số hạn chế cần được cải thiện trong tương lai:

(1) Hệ thống mới triển khai trên mạng thử nghiệm, chưa áp dụng trên mạng chính (mainnet), do đó chưa đánh giá đầy đủ về chi phí gas và hiệu năng thực tế.

(2) Chưa tích hợp các cơ chế bảo mật nâng cao như kiểm tra lỗ hổng Smart Contract (reentrancy, overflow) [8] hoặc xác thực đa lớp cho người dùng.

(3) Giao diện người dùng còn đơn giản, chưa tối ưu trải nghiệm người dùng (UX) và chưa hỗ trợ nhiều tính năng nâng cao như tìm kiếm, phân loại chiến dịch.

(4) Trong tương lai, hệ thống có thể được mở rộng với các chức năng như tích hợp nhiều loại token (ERC-20), triển khai trên nhiều mạng blockchain khác nhau, tối ưu chi phí gas và nâng cao hiệu suất xử lý [7].

Tóm lại, đề tài đã đạt được mục tiêu đề ra, xây dựng thành công một hệ thống DApp gây quỹ cơ bản, góp phần minh họa rõ ràng tiềm năng của công nghệ Blockchain trong việc nâng cao tính minh bạch và độ tin cậy trong các hoạt động tài chính [6]. Đây cũng là nền tảng quan trọng để tiếp tục nghiên cứu và phát triển các ứng dụng thực tiễn trong tương lai.

TÀI LIỆU THAM KHẢO

Tiếng Việt

[1] **Huỳnh Minh Châu** (2026), "Thực hành ngôn ngữ lập trình Solidity: Từ biến cơ bản đến Struct và Mapping," *Tài liệu thực hành tổng hợp*, Trường Đại học Đồng Tháp.

[2] **Huỳnh Minh Châu** (2026), "Hướng dẫn lập trình Smart Contract nâng cao: Ticketing Solution và Quản lý trạng thái," *Tài liệu giảng dạy*, Trường Đại học Đồng Tháp.

[3] **Huỳnh Minh Châu** (2026), "Kỹ thuật Upgradeable Smart Contract bằng OpenZeppelin CLI," *Tài liệu hướng dẫn thực hành chuyên sâu.*

[4] **Huỳnh Minh Châu** (2026), "Quy trình viết Unit Test cho Smart Contract với Mocha và Chai," *Tài liệu hướng dẫn kỹ thuật.*

Tiếng Anh

[5] Gavin Zheng, Longxiang Gao, Liqun Huang, Jian Guan (2021), *Ethereum Smart Contract Development in Solidity*, Springer Singapore. doi: 10.1007/978-981-15-6218-1

[6] Andreas M. Antonopoulos, Gavin Wood (2018), *Mastering Ethereum: Building Smart Contracts and DApps*, O'Reilly Media.

[7] OpenZeppelin Docs, "Proxy Patterns and Upgradeable Contracts," [Online]. Available: https://docs.openzeppelin.com/ (Accessed: April 2026).

[8] Ethereum Foundation, "Smart Contract Security and Testing," [Online]. Available: https://ethereum.org/ (Accessed: April 2026).